

Tổng Hợp Kiến Thức

1. Express.js Là Gì?

- **Framework tối giản** cho Node.js, xây dựng trên module `http`.
- **Công dụng:** Đơn giản hóa phát triển web/API với routing, middleware, response methods.
- **Lợi ích:** Code ngắn gọn, dễ quản lý hơn so với `http` thuần.

2. Thiết Lập Dự Án Express.js

- **Khởi tạo dự án:**

```
mkdir express-api-demo
cd express-api-demo
npm init -y
npm install express
```

- **Tạo file `server.js`:**

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Chào mừng đến với Express API!');
});

app.listen(PORT, () => {
  console.log(`Server chạy tại <http://localhost>:${PORT}`);
});
```

- **Chạy server:**

```
node server.js # hoặc nodemon server.js (nếu cài nodemon)
```

Giải Thích Cơ Bản

- `app.get(path, callback)`: Định nghĩa route GET.
- `req`: Request object (dữ liệu từ client).
- `res`: Response object (gửi dữ liệu về client).
- `app.listen(port)`: Khởi động server.

3. Cấu Trúc Dự Án

Thư Mục Đề Xuất

```
express-api-demo/
├── node_modules/    # Dependencies
├── routes/          # Chứa các route
│   └── productRoutes.js
├── middlewares/     # Chứa middleware
│   ├── requestLogger.js
│   └── errorHandler.js
├── server.js        # File chính
└── data.json        # Dữ liệu mẫu
```

```
├── package.json
├── package-lock.json
```

Modules trong Node.js

- **CommonJS (CJS):**

- Import: `const module = require('./path');`
- Export: `module.exports = moduleName;`

- **Ví dụ:**

- `routes/productRoutes.js` :

```
const express = require('express');
const router = express.Router();
router.get('/products', (req, res) => {
  res.json([{ id: 1, name: 'Sản phẩm 1' }]);
});
module.exports = router;
```

- `server.js` :

```
const productsRoutes = require('./routes/productRoutes');
app.use('/api', productsRoutes); // Mount routes
```

4. Xây Dựng RESTful API (CRUD)

Middleware Quan Trọng

- `app.use(express.json())` : Parse request body dạng JSON.

Routes CRUD

- **File** `routes/productRoutes.js` :

```
const express = require('express');
const router = express.Router();
const fs = require('fs').promises;

// GET /api/products
router.get('/products', async (req, res, next) => {
  try {
    const data = await fs.readFile('data.json', 'utf8');
    res.json(JSON.parse(data));
  } catch (err) { next(err); }
});

// GET /api/products/:id
router.get('/products/:id', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);
    const data = await fs.readFile('data.json', 'utf8');
    const products = JSON.parse(data);
    const product = products.find(p => p.id === id);
    if (!product) return res.status(404).send('Không tìm thấy');
    res.json(product);
  } catch (err) { next(err); }
});

// POST /api/products
```

```

router.post('/products', async (req, res, next) => {
  try {
    const newProduct = req.body;
    const data = await fs.readFile('data.json', 'utf8');
    const products = JSON.parse(data);
    newProduct.id = products.length + 1;
    products.push(newProduct);
    await fs.writeFile('data.json', JSON.stringify(products, null, 2));
    res.status(201).json(newProduct);
  } catch (err) { next(err); }
});

// PUT /api/products/:id
router.put('/products/:id', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);
    const updatedProduct = req.body;
    const data = await fs.readFile('data.json', 'utf8');
    let products = JSON.parse(data);
    const index = products.findIndex(p => p.id === id);
    if (index === -1) return res.status(404).send('Không tìm thấy');
    products[index] = { ...products[index], ...updatedProduct, id };
    await fs.writeFile('data.json', JSON.stringify(products, null, 2));
    res.json(products[index]);
  } catch (err) { next(err); }
});

// DELETE /api/products/:id
router.delete('/products/:id', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);
    const data = await fs.readFile('data.json', 'utf8');
    let products = JSON.parse(data);
    const index = products.findIndex(p => p.id === id);
    if (index === -1) return res.status(404).send('Không tìm thấy');
    products.splice(index, 1);
    await fs.writeFile('data.json', JSON.stringify(products, null, 2));
    res.status(204).send();
  } catch (err) { next(err); }
});

module.exports = router;

```

Gắn Routes vào `server.js` :

```

const productsRoutes = require('./routes/productRoutes');
app.use('/api', productsRoutes); // Base path: /api

```

Response Methods

- `res.send(data)` : Gửi dữ liệu (text/html mặc định).
- `res.json(data)` : Gửi dữ liệu JSON.
- `res.status(code)` : Đặt mã trạng thái HTTP (200, 404, 500...).

5. Middleware

Khái Niệm

- **Middleware:** Hàm xử lý request trước route handler, có `req` , `res` , `next` .
- `next()` : Chuyển quyền điều khiển sang middleware/route tiếp theo.
- **Thứ tự:** Middleware chạy theo thứ tự khai báo trong `app.use()` .

Ví dụ 1: Logging Request

- `middlewares/requestLogger.js` :

```
function requestLoggerMiddleware(req, res, next) {  
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);  
  next();  
}  
module.exports = requestLoggerMiddleware;
```

- Sử dụng trong `server.js` :

```
const requestLoggerMiddleware = require('./middlewares/requestLogger');  
app.use(requestLoggerMiddleware); // Trước routes
```

Ví dụ 2: Error Handling

- `middlewares/errorHandler.js` :

```
function errorHandlerMiddleware(err, req, res, next) {  
  console.error('ERROR:', err.stack);  
  res.status(500).send('Lỗi server');  
}  
module.exports = errorHandlerMiddleware;
```

- Sử dụng trong `server.js` :

```
const errorHandlerMiddleware = require('./middlewares/errorHandler');  
app.use(errorHandlerMiddleware); // Sau routes
```

- **Chú ý:** Đặt **cuối cùng** trong chuỗi middleware.

Thứ Tự Middleware trong `server.js`

```
app.use(requestLoggerMiddleware); // 1. Log request  
app.use(express.json()); // 2. Parse JSON  
app.use('/api', productsRoutes); // 3. Routes  
app.use(errorHandlerMiddleware); // 4. Xử lý lỗi (cuối cùng)
```